DUMPSTER — Native SwiftUI Conversion Specification

Version: 1.0

Target Audience: Claude (Anthropic AI) — to be executed as a coding task

Scope: Convert the Dumpster iOS app from a WKWebView-embedded React/Vite web app to a 100% native SwiftUI implementation

---

Table of Contents

1. Executive Summary
2. Current Architecture Map
3. Files to Delete
4. Files to Preserve Unchanged
5. Files to Modify
6. New Files to Create
7. Data Model (SwiftData)
8. App State Object
9. Design Tokens & Theming
10. View Hierarchy & Screen Layout
11. PhotoCardView — Complete Specification
12. DumpCardView — Complete Specification
13. PhotoPoolView — Complete Specification
14. CaptionPoolView — Complete Specification
15. LightboxView — Complete Specification
16. Formula Engine (Port from TypeScript)
17. Undo/Redo System
18. JavaScript Bridge Replacement Map
19. AISuggestView Integration Changes
20. DumpsterApp Entry Point Changes
21. ContentView Rewrite

22. Pending Features — Full Specifications
23. Animations & Transitions Reference
24. Haptic Feedback Map
25. Photo Storage Strategy
26. Xcode Project Changes
27. Step-by-Step Execution Order

---

1. Executive Summary

Dumpster is an iOS app for curating Instagram carousel photo dumps. It currently wraps a React/Vite web app inside a `WKWebView` using a custom `dumpster://` URL scheme handler. Native Swift code handles AI features (Apple Vision photo classification via `PhotoAnalyzer`, multi-provider LLM captions via `LLMService`), in-app purchases (`CreditManager` / `PaywallView`), and a file-cabinet settings menu (`FileCabinetMenuView`). A JavaScript bridge (`dumpsterBridge` message handler) connects the two layers.

The goal is to **completely remove** the WKWebView, the `dist/` bundle, and the `DumpsterSchemeHandler`, and replace the entire web UI with native SwiftUI views — while preserving every existing native feature and adding several pending features that are spec'd but not yet built.

The app targets **iOS 17.0+** and uses **SwiftData** for persistence.

---

2. Current Architecture Map

iOS Native Files (in `ios/DumpsterIOS/`)

File	Role	Action
`DumpsterApp.swift`	App entry point (`@main`)	**Modify** — add `ModelContainer`
`ContentView.swift`	Root view + WKWebView + AppState + Dynamic Island + CabinetMenuView + CaptionOptionButton + Color extension	**Heavy rewrite** — remove WKWebView, keep Dynamic Island, replace web content with native views
`DumpsterSchemeHandler.swift`	Custom `dumpster://` URL scheme handler serving `dist/` files	**Delete entirely**

`AISuggestView.swift`	AI dump builder (PhotosPicker → Vision → clusters → creates dumps via JS bridge)	**Modify** — replace JS bridge call with direct SwiftData insertion
`PhotoAnalyzer.swift`	Apple Vision image classification and clustering	**Preserve unchanged**
`LLMService.swift`	Multi-provider LLM service (OpenAI, Claude, Gemini, Manus, Perplexity)	**Preserve unchanged**
`CaptionService.swift`	Backward-compat wrapper around LLMService	**Preserve unchanged**
`CreditManager.swift`	Credit balance + StoreKit 2 IAP	**Preserve unchanged**
`PaywallView.swift`	IAP paywall sheet + CreditBadge view	**Preserve unchanged**
`FileCabinetMenuView.swift`	Full-screen file cabinet menu with 5 tab folders	**Preserve unchanged**
`SettingsView.swift`	Legacy settings redirect	**Preserve unchanged**

Web App Files (at repo root `src/` and `dumpster/src/`)

These are **reference only** — they define the features to replicate natively. They will not be included in the iOS bundle after conversion.

File	What it defines
`src/types.ts`	`Photo`, `Dump`, `Caption`, `Filter`, `ColorMode`, `PoolSize` types
`src/store.ts`	Zustand store: all state, mutations, persistence, undo/redo, AI bridge
`src/formula.ts`	Carousel arrangement heuristics, slot scoring, title generation, vibe check
`src/App.tsx`	Root layout: header, dump list, action row, pool toggle, stats
`src/components/DumpCard.tsx`	Per-dump carousel with drag-and-drop reordering

`src/components/PhotoCard.tsx`	Photo tile with gestures, menu, visual states
`src/components/PhotoPool.tsx`	Photo grid with filters, search, upload, size toggle
`src/components/CaptionPool.tsx`	Caption library with styles, generation, filtering
`src/components/Lightbox.tsx`	Full-screen photo enlargement modal

---

3. Files to Delete

1. **`ios/DumpsterIOS/DumpsterSchemeHandler.swift`** — entire file.
2. **`dist/` folder** — the bundled React build output currently referenced in the Xcode project's Resources build phase. Remove the folder reference from the project navigator and delete the copy-resources build phase entry.
3. **`WebKit.framework`** — remove from the target's "Frameworks, Libraries, and Embedded Content" in Xcode. (The `Vision.framework` must remain.)

---

4. Files to Preserve Unchanged

These files require **zero modifications**:

- `PhotoAnalyzer.swift`
- `LLMService.swift`
- `CaptionService.swift`
- `CreditManager.swift`
- `PaywallView.swift` (including `CreditBadge`)
- `FileCabinetMenuView.swift`
- `SettingsView.swift`
- `Assets.xcassets` (app icon, launch background)
- `Info.plist`

---

5. Files to Modify

5.1. `DumpsterApp.swift`

Add the SwiftData `ModelContainer` for `Photo`, `Dump`, and `Caption`.

Swift

```
import SwiftUI
import SwiftData

@main
struct DumpsterApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
                .ignoresSafeArea()
        }
        .modelContainer(for: [DumpPhoto.self, PhotoDump.self, DumpCaption.self]
    }
}
```

> **Note on naming:** To avoid collisions with SwiftUI's built-in types, use `DumpPhoto`, `PhotoDump`, and `DumpCaption` as the SwiftData model names. The spec uses these names throughout.

5.2. `ContentView.swift`

Heavy rewrite. See Section 21 for the complete replacement.

5.3. `AISuggestView.swift`

Targeted modification. See Section 19.

---

6. New Files to Create

File	Purpose
`Models/DumpPhoto.swift`	SwiftData `@Model` for a photo
`Models/PhotoDump.swift`	SwiftData `@Model` for a dump
`Models/DumpCaption.swift`	SwiftData `@Model` for a caption
`Views/MainAppView.swift`	Scrollable main content (header, dumps, actions, pool)
`Views/DumpCardView.swift`	Single dump carousel card

`Views/PhotoCardView.swift`	Reusable photo tile (pool and dump contexts)
`Views/PhotoPoolView.swift`	Photo grid with filters, search, upload
`Views/CaptionPoolView.swift`	Caption library panel
`Views/LightboxView.swift`	Full-screen photo enlargement overlay
`Views/PoolFilterMenu.swift`	Hamburger filter menu (slides right)
`Views/DumpCaptionBubble.swift`	Caption generation bubble (10 options, scrollable)
`Utilities/FormulaEngine.swift`	Port of `formula.ts` — arrangement, scoring, title gen, vibe check
`Utilities/UndoManager.swift`	Snapshot-based undo/redo stack
`Utilities/PhotoStorageManager.swift`	Save/load photos to app's Documents directory
`Utilities/Theme.swift`	Design tokens and theming

---

7. Data Model (SwiftData)

7.1. `DumpPhoto`

Swift

```swift
import Foundation
import SwiftData

@Model
final class DumpPhoto {
    @Attribute(.unique) var id: String
    var localPath: String       // Relative path within app's Documents/photos/
    var filename: String
    var category: String        // e.g., "AUTOMOTIVE", "PORTRAIT", "NIGHTLIFE"
    var labels: [String]        // Free-text labels, multiple per photo
    var starred: Bool
    var isHuji: Bool
    var createdAt: Date

    /// Computed: full URL to the image file
    var imageURL: URL {
```

```
        let docs = FileManager.default.urls(for: .documentDirectory, in: .userD
        return docs.appendingPathComponent(localPath)
    }

    init(
        id: String = "p-\(Date.now.timeIntervalSince1970)-\(UUID().uuidString.p
        localPath: String,
        filename: String,
        category: String = "LIFESTYLE",
        labels: [String] = [],
        starred: Bool = false,
        isHuji: Bool = false
    ) {
        self.id = id
        self.localPath = localPath
        self.filename = filename
        self.category = category
        self.labels = labels
        self.starred = starred
        self.isHuji = isHuji
        self.createdAt = Date()
    }
}
```

7.2. PhotoDump

Swift

```
import Foundation
import SwiftData

@Model
final class PhotoDump {
    @Attribute(.unique) var id: String
    var num: Int
    var title: String
    var photoIDs: [String]      // Ordered array of DumpPhoto.id values. Max 20
    var vibeBadge: String?      // nil or "mismatch"
    var liked: Bool
    var titleApproved: Bool?    // true = user kept it, false = rejected, nil =
    var createdAt: Date

    init(
        id: String = "d-\(Date.now.timeIntervalSince1970)",
        num: Int,
        title: String = "Untitled Dump",
        photoIDs: [String] = [],
```

```
        vibeBadge: String? = nil,
        liked: Bool = false,
        titleApproved: Bool? = nil
    ) {
        self.id = id
        self.num = num
        self.title = title
        self.photoIDs = photoIDs
        self.vibeBadge = vibeBadge
        self.liked = liked
        self.titleApproved = titleApproved
        self.createdAt = Date()
    }
}
```

7.3. DumpCaption

Swift

```
import Foundation
import SwiftData

@Model
final class DumpCaption {
    @Attribute(.unique) var id: String
    var text: String
    var style: String           // "storytelling", "emoji", "clean", "numbered"
    var rating: Int             // 0–5
    var dumpId: String?         // Associated PhotoDump.id, or nil for general
    var createdAt: Date
    var favorited: Bool
    var banned: Bool            // Thumbs-down = never use again

    init(
        id: String = "cap-\(Date.now.timeIntervalSince1970)-\(UUID().uuidString
        text: String,
        style: String = "clean",
        rating: Int = 0,
        dumpId: String? = nil,
        favorited: Bool = false,
        banned: Bool = false
    ) {
        self.id = id
        self.text = text
        self.style = style
        self.rating = rating
        self.dumpId = dumpId
```

```
        self.createdAt = Date()
        self.favorited = favorited
        self.banned = banned
    }
}
```

---

8. App State Object

Rewrite `AppState` in `ContentView.swift`. Remove all `WKWebView`-related properties and methods. Add native UI state.

Swift

```
import SwiftUI

@MainActor
final class AppState: ObservableObject {
    // ── Overlay toggles ──
    @Published var showAISuggest = false
    @Published var showSettings = false
    @Published var showFileCabinet = false

    // ── Dynamic Island ──
    @Published var statusText: String = "DUMPSTER"
    @Published var dumpCount: Int = 0
    @Published var isAnalyzing = false

    // ── AI results (used by CabinetMenuView / AISuggestView) ──
    @Published var captionResults: [LLMService.CaptionResult] = []

    // ── UI state (replaces Zustand store's UI fields) ──
    @Published var activeDumpId: String?
    @Published var colorMode: ColorMode = .dark {
        didSet { UserDefaults.standard.set(colorMode.rawValue, forKey: "dumpste
    }
    @Published var poolSize: PoolSize = .large {
        didSet { UserDefaults.standard.set(poolSize.rawValue, forKey: "dumpster_
    }
    @Published var activeFilters: Set<FilterType> = []
    @Published var poolSearchQuery: String = ""
    @Published var lightboxPhotoId: String?
    @Published var addingToDumpId: String?       // When non-nil, pool is in "s
    @Published var activePoolTab: PoolTab = .photos

    // ── Enums ──
    enum FilterType: String, CaseIterable, Identifiable {
```

```
        case all, starred, huji, used, videos
        var id: String { rawValue }
    }
    enum ColorMode: String, CaseIterable {
        case dark, day, system
    }
    enum PoolSize: String, CaseIterable {
        case small, medium, large

        var columnCount: Int {
            switch self {
            case .small: return 6
            case .medium: return 4
            case .large: return 2
            }
        }
    }
    enum PoolTab: String { case photos, captions }

    // ── Status helper ──
    func showStatus(_ text: String, duration: TimeInterval = 3.0) {
        statusText = text
        Task { @MainActor in
            try? await Task.sleep(for: .seconds(duration))
            if statusText == text { statusText = "DUMPSTER" }
        }
    }

    // ── Init: load persisted preferences ──
    init() {
        if let cm = UserDefaults.standard.string(forKey: "dumpster_colorMode"),
           let mode = ColorMode(rawValue: cm) {
            colorMode = mode
        }
        if let ps = UserDefaults.standard.string(forKey: "dumpster_poolSize"),
           let size = PoolSize(rawValue: ps) {
            poolSize = size
        }
    }
}
```

Critical: Remove these properties/methods that currently exist in `AppState`:

- `webView: WKWebView?`
- `evaluateJS(_:completion:)`
- `sendCaptionsToWeb(_:)`

9. Design Tokens & Theming

Create `Utilities/Theme.swift`:

Swift

```
import SwiftUI

struct Theme {
    // ── Primary Backgrounds ──
    static func bg(_ mode: AppState.ColorMode, _ cs: ColorScheme) -> Color {
        isDark(mode, cs) ? Color(hex: "#0A0A0A") : Color(hex: "#F0EBE0")
    }
    static func bg1(_ mode: AppState.ColorMode, _ cs: ColorScheme) -> Color {
        isDark(mode, cs) ? Color(hex: "#111111") : Color(hex: "#E8E2D5")
    }
    static func bg2(_ mode: AppState.ColorMode, _ cs: ColorScheme) -> Color {
        isDark(mode, cs) ? Color(hex: "#181818") : Color(hex: "#DFD8C8")
    }
    static func bg3(_ mode: AppState.ColorMode, _ cs: ColorScheme) -> Color {
        isDark(mode, cs) ? Color(hex: "#222222") : Color(hex: "#D4CCBA")
    }

    // ── Borders ──
    static func border(_ mode: AppState.ColorMode, _ cs: ColorScheme) -> Color
        isDark(mode, cs) ? Color(hex: "#1E1E1E") : Color(hex: "#C8BFA8")
    }
    static func border2(_ mode: AppState.ColorMode, _ cs: ColorScheme) -> Color
        isDark(mode, cs) ? Color(hex: "#2A2A2A") : Color(hex: "#B8AE98")
    }

    // ── Text ──
    static func text(_ mode: AppState.ColorMode, _ cs: ColorScheme) -> Color {
        isDark(mode, cs) ? Color(hex: "#E8E8E8") : Color(hex: "#1A1610")
    }
    static func text2(_ mode: AppState.ColorMode, _ cs: ColorScheme) -> Color {
        isDark(mode, cs) ? Color(hex: "#999999") : Color(hex: "#5A5040")
    }
    static func text3(_ mode: AppState.ColorMode, _ cs: ColorScheme) -> Color {
        isDark(mode, cs) ? Color(hex: "#555555") : Color(hex: "#8A7A60")
    }

    // ── Accent Colors ──
    static let gold = Color(hex: "#C8A96E")
    static let goldDim = Color(hex: "#C8A96E").opacity(0.15)
    static let hujiOutline = Color.red
    static let starBadge = Color(hex: "#C8A96E")
```

```
    static let removeText = Color.red

    // ── Helper ──
    static func isDark(_ mode: AppState.ColorMode, _ cs: ColorScheme) -> Bool {
        mode == .dark || (mode == .system && cs == .dark)
    }
}
```

Safe area insets: All content must respect the iPhone notch / Dynamic Island. The Dynamic Island capsule overlay sits at the top with `.padding(.top, 13)` and the main scrollable content should use `safeAreaInset(edge: .top)` of approximately 60pt to clear the capsule.

---

10. View Hierarchy & Screen Layout

The view tree mirrors the web app's `App.tsx`:

Plain Text

```
DumpsterApp
└── ContentView (ZStack)
    ├── MainAppView (ScrollView, base layer)
    │   ├── HeaderView
    │   │   ├── Logo + "DUMPSTER" wordmark
    │   │   ├── Undo / Redo buttons
    │   │   └── Stats pills (Dumps, Photos Used, In Pool)
    │   ├── Divider
    │   ├── ForEach(dumps) → DumpCardView
    │   │   ├── DumpHeader (num, title, vibe badge, like, share, delete)
    │   │   ├── Horizontal ScrollView of PhotoCardView slots
    │   │   │   ├── PhotoCardView (per photo in dump)
    │   │   │   ├── "+" Add Photos card
    │   │   │   └── Upload card
    │   │   └── Progress bar (fill / 20, gold in 10-12 peak zone)
    │   ├── Action Row
    │   │   ├── "+ New Dump" button
    │   │   └── "Auto-Generate" button (with count picker popover)
    │   ├── Divider
    │   ├── Pool Toggle (Photos / Captions segmented pill)
    │   └── Pool Area
    │       ├── PhotoPoolView (LazyVGrid of PhotoCardView)
    │       │   ├── Pool header + filter hamburger + search + size toggles
    │       │   ├── Active filter chips
    │       │   ├── Photo grid
    │       │   └── "Add Photos" upload card (at end)
    │       └── CaptionPoolView
    │           ├── Style chips + auto-generate button
```

```
    │           ├── Custom text entry
    │           ├── Filter tabs (All / Favorites / Banned)
    │           └── Caption list
    ├── Dynamic Island capsule (top overlay, ZStack layer 2)
    ├── CabinetMenuView (AI Suggestions bottom sheet, ZStack layer 3)
    ├── FileCabinetMenuView (full-screen overlay, ZStack layer 4)
    ├── LightboxView (full-screen overlay, ZStack layer 5)
    └── .fullScreenCover → SettingsView
```

---

11. PhotoCardView — Complete Specification

Create `Views/PhotoCardView.swift`. This is the core reusable component used in both the pool grid and dump carousels.

11.1. Props / Parameters

Swift

```swift
struct PhotoCardView: View {
    let photo: DumpPhoto
    let context: CardContext       // .pool or .dump(dumpId: String)
    let isSelected: Bool           // Green border in add-to-dump mode
    let isUsed: Bool               // Dimmed + checkmark in pool
    let slotIndex: Int?            // Position in dump (nil for pool)
    let totalInDump: Int?          // Total photos in this dump
    let size: CGSize               // Computed from pool size or dump slot size

    var onTap: (() -> Void)?
    var onDoubleTap: (() -> Void)?
    var onRemoveFromDump: (() -> Void)?
    var onToggleStar: (() -> Void)?
    var onToggleHuji: (() -> Void)?
    var onOpenLightbox: (() -> Void)?

    enum CardContext {
        case pool
        case dump(dumpId: String)
    }
}
```

11.2. Visual States

State	Visual Treatment

Default	Rounded rectangle (cornerRadius 10), image fills frame
Huji marked	Thin **red** border (2pt), small "HUJI" badge top-right
Starred	Small **gold star** badge top-left
Selected (add mode)	**Green** border (2pt), green overlay at 20% opacity, checkmark
Used (in pool)	Dimmed (40% opacity overlay), small checkmark badge
Dragging	**Gold** border (2pt), slight scale-up (1.05)
First in dump (Hook)	No special label or outline — importance is purely positional

11.3. Gesture Handling

Single Tap: Detected via `.onTapGesture(count: 1)`. Behavior depends on context:

- If `appState.addingToDumpId != nil` (pool is in selection mode): toggle selection.
- If in dump context: show the "..." dots menu (set a `@State var showMenu = true`).
- If in pool context with no active selection: show the "..." dots menu.

Double Tap: Detected via `.onTapGesture(count: 2)`. Always opens the lightbox by setting `appState.lightboxPhotoId = photo.id`. The double-tap gesture must be added **before** the single-tap gesture so SwiftUI can disambiguate.

Long Press + Drag (dump context only): For reordering within a dump. Use `.onLongPressGesture(minimumDuration: 0.5)` combined with `.draggable(photo.id)` and `.dropDestination(for: String.self)` on the parent container. During drag, apply a scale of 1.05 and gold border.

11.4. "..." Dots Menu

When `showMenu == true`, display a small overlay menu anchored to the photo card:

Swift

```
.confirmationDialog("", isPresented: $showMenu) {
    Button("Mark as Huji") { onToggleHuji?() }      // or "Unmark Huji" if alre
    Button("Favorite") { onToggleStar?() }            // or "Unfavorite"
    Button("Open Lightbox") { onOpenLightbox?() }
```

```
    if case .dump = context {
        Button("Remove", role: .destructive) { onRemoveFromDump?() }
    }
}
```

The web app shows "Remove" in **red font** — using `role: .destructive` achieves this natively.

11.5. Bottom Gradient & Category Label

A subtle gradient overlay at the bottom of the photo shows the category text (e.g., "AUTOMOTIVE") in small caps. In the web app this is editable — for native, make it tappable to show a picker or text field.

---

12. DumpCardView — Complete Specification

Create `Views/DumpCardView.swift`.

12.1. Structure

Swift

```
struct DumpCardView: View {
    let dump: PhotoDump
    let photos: [DumpPhoto]     // Resolved from dump.photoIDs
    let isActive: Bool
    @EnvironmentObject var appState: AppState
    @Environment(\.modelContext) var modelContext
}
```

12.2. Header Row

- **Left:** Dump number badge (e.g., "01"), editable title as `TextField`.
- **Center:** Vibe badge if `dump.vibeBadge == "mismatch"` — show a yellow warning icon.
- **Right:** Like button (heart, toggles `dump.liked`), Share button, Delete button (trash icon, with confirmation alert).

Title editing: The title is a `TextField` that looks like plain text. On tap, it becomes editable. On submit (Return key), save the new title. This implements the "tap to rename dump" pending feature.

Title approve/reject: If `dump.titleApproved == nil`, show small checkmark and X buttons next to the title. Checkmark sets `titleApproved = true`. X sets `titleApproved = false` and triggers `LLMService.shared.generateTitle(...)` to regenerate.

12.3. Horizontal Photo Carousel

Swift

```
ScrollView(.horizontal, showsIndicators: false) {
    HStack(spacing: 8) {
        ForEach(Array(photos.enumerated()), id: \.element.id) { index, photo in
            PhotoCardView(
                photo: photo,
                context: .dump(dumpId: dump.id),
                isSelected: false,
                isUsed: false,
                slotIndex: index,
                totalInDump: photos.count,
                size: CGSize(width: 120, height: 160)
            )
        }

        // "+" Add Photos card (if < 20 photos)
        if photos.count < 20 {
            addPhotosCard
        }
    }
    .padding(.horizontal, 16)
}
```

Drag to reorder: Wrap the `HStack` in a drag-and-drop context. When a photo is dropped at a new position, update `dump.photoIDs` by removing the dragged ID and inserting it at the drop index. Push an undo snapshot before the mutation.

The "+" card: A rounded rectangle with a "+" icon and "Add Photos" text. On tap, set `appState.addingToDumpId = dump.id` and scroll to the photo pool section using `ScrollViewReader`.

12.4. Progress Bar Footer

A thin horizontal bar showing `photos.count / 20` fill. The bar turns **gold** when count is in the 10–12 "peak zone" range, otherwise it's a subtle gray.

Swift

```
GeometryReader { geo in
    let fraction = CGFloat(photos.count) / 20.0
    let isPeak = photos.count >= 10 && photos.count <= 12
    RoundedRectangle(cornerRadius: 2)
        .fill(isPeak ? Theme.gold : Color.white.opacity(0.15))
```

```
        .frame(width: geo.size.width * fraction, height: 3)
}
.frame(height: 3)
```

12.5. Remove from Dump Animation

When a photo is removed from a dump, it should animate back to the pool. Implement this as:

1. The photo card scales down to 0.8 and fades to 0 over 0.3s.
2. The remaining cards slide left to fill the gap with `.animation(.spring())`.
3. The photo remains in the `DumpPhoto` table (it is NOT deleted) — only its ID is removed from `dump.photoIDs`.

---

13. PhotoPoolView — Complete Specification

Create `Views/PhotoPoolView.swift`.

13.1. Header

Plain Text

```
PHOTO POOL                    [hamburger] [search] [+] [-]
```

- **"PHOTO POOL"** title in gold, small caps, tracking 2.
- **Hamburger icon:** Toggles `PoolFilterMenu` (slides in from the left or expands right).
- **Search icon (magnifying glass):** Toggles a `TextField` bound to `appState.poolSearchQuery`.
- **+/- buttons:** Cycle `appState.poolSize` between `.small`, `.medium`, `.large`. These are subtle, small buttons.

13.2. Active Filter Chips

When `appState.activeFilters` is non-empty, show a horizontal `ScrollView` of filter chips. Each chip shows the filter name and an "x" to remove it.

13.3. Grid

Swift

```
let columns = Array(repeating: GridItem(.flexible(), spacing: 6), count: appSta
```

```
LazyVGrid(columns: columns, spacing: 6) {
    ForEach(filteredPhotos) { photo in
        PhotoCardView(
            photo: photo,
            context: .pool,
            isSelected: selectedPhotoIDs.contains(photo.id),
            isUsed: usedPhotoIDs.contains(photo.id),
            slotIndex: nil,
            totalInDump: nil,
            size: computedSize
        )
    }

    // "Add Photos" upload card at end
    AddPhotosCard()
}
```

13.4. Filtering Logic

Compute `filteredPhotos` from the full photo list:

Swift

```
var filteredPhotos: [DumpPhoto] {
    var result = allPhotos

    // Apply active filters
    for filter in appState.activeFilters {
        switch filter {
        case .starred: result = result.filter { $0.starred }
        case .huji:    result = result.filter { $0.isHuji }
        case .used:    result = result.filter { usedPhotoIDs.contains($0.id) }
        case .videos:  result = result.filter { isVideo($0.filename) }
        case .all:     break
        }
    }

    // Apply search query (searches labels, category, filename)
    if !appState.poolSearchQuery.isEmpty {
        let q = appState.poolSearchQuery.lowercased()
        result = result.filter {
            $0.filename.lowercased().contains(q) ||
            $0.category.lowercased().contains(q) ||
            $0.labels.contains { $0.lowercased().contains(q) }
        }
    }
```

```
    return result
}
```

13.5. "Add to Dump" Selection Mode

When `appState.addingToDumpId` is non-nil, the pool enters selection mode:

- A banner appears at the top: "Select photos to add to [Dump Title]" with a "Done" button.
- Tapping a photo toggles it in a local `@State var selectedPhotoIDs: Set<String>`.
- "Done" calls `addPhotosToDump(selectedPhotoIDs, dumpId)`, clears selection, and sets `appState.addingToDumpId = nil`.

13.6. Upload Card

The "Add Photos" card at the end of the grid triggers a `PhotosPicker` (from `PhotosUI`). Selected images are saved to the app's Documents directory via `PhotoStorageManager` and inserted as `DumpPhoto` records.

13.7. Pinch to Resize

Add a `MagnificationGesture` to the grid area. On gesture end, if scale > 1.3, increase pool size (fewer columns). If scale < 0.7, decrease pool size (more columns).

---

14. CaptionPoolView — Complete Specification

Create `Views/CaptionPoolView.swift`.

14.1. Structure

The caption pool is shown when `appState.activePoolTab == .captions`.

Style Chips: Horizontal row of selectable style chips: "Storytelling", "Emoji", "Clean", "Numbered".

Auto-Generate Button: "Auto-Generate [Style]" — picks a non-duplicate caption from a hardcoded template bank for the selected style and saves it as a `DumpCaption` with `dumpId = appState.activeDumpId`.

Custom Text Entry: A `TextField` with a submit button to add a manual caption.

Filter Tabs: "All", "Favorites", "Banned" — filter the caption list.

Caption List: Each `DumpCaption` rendered as a card with:

- Caption text

- Copy button (copies to clipboard via `UIPasteboard.general.string`)
- Thumbs-up button (toggles `favorited`)
- Thumbs-down button (sets `banned = true`, `favorited = false`)
- Delete button
- Banned captions: red-tinted background, strikethrough text, "Never Use" badge

14.2. Template Banks (from `CaptionPool.tsx`)

Swift

```
static let templateBanks: [String: [String]] = [
    "storytelling": [
        "the kind of night you tell stories about",
        "we didn't plan this, it just happened",
        "somewhere between the chaos and the calm",
        "a collection of moments I refuse to forget",
        "the unfiltered version of last week",
        // ... (include all from web source)
    ],
    "emoji": [
        "📸✨🔥", "🌙💫🖤", "🏎️💨✨",
        // ...
    ],
    "clean": [
        "recent.", "documented.", "filed under: good times",
        // ...
    ],
    "numbered": [
        "1. showed up  2. showed out", "1/10 of why this week hit different",
        // ...
    ]
]
```

---

15. LightboxView — Complete Specification

Create `Views/LightboxView.swift`.

Shown when `appState.lightboxPhotoId != nil`. A full-screen overlay.

Swift

```
struct LightboxView: View {
    @EnvironmentObject var appState: AppState
```

```
    @Environment(\.modelContext) var modelContext

    var body: some View {
        if let photoId = appState.lightboxPhotoId,
           let photo = fetchPhoto(id: photoId) {
            ZStack {
                // Blurred dark backdrop
                Color.black.opacity(0.9)
                    .ignoresSafeArea()
                    .onTapGesture { appState.lightboxPhotoId = nil }

                VStack {
                    // Close button (top-right)
                    HStack {
                        Spacer()
                        Button { appState.lightboxPhotoId = nil } label: {
                            Image(systemName: "xmark.circle.fill")
                                .font(.system(size: 28))
                                .foregroundColor(.white.opacity(0.7))
                        }
                        .padding(24)
                    }

                    Spacer()

                    // Full-size image
                    if let uiImage = UIImage(contentsOfFile: photo.imageURL.pat
                        Image(uiImage: uiImage)
                            .resizable()
                            .scaledToFit()
                            .clipShape(RoundedRectangle(cornerRadius: 12))
                            .padding(.horizontal, 16)
                    }

                    // Info strip
                    HStack(spacing: 12) {
                        Text(photo.category)
                            .font(.system(size: 11, weight: .bold))
                            .tracking(1)
                            .foregroundColor(Theme.gold)
                        Text(photo.filename)
                            .font(.system(size: 11))
                            .foregroundColor(.white.opacity(0.4))
                        if photo.isHuji {
                            Text("HUJI")
                                .font(.system(size: 9, weight: .bold))
                                .foregroundColor(.red)
                                .padding(.horizontal, 6)
```

```
                                .padding(.vertical, 2)
                                .background(Color.red.opacity(0.15))
                                .cornerRadius(4)
                        }
                    }
                    .padding(.top, 12)

                    Spacer()
                }
            }
            .transition(.opacity)
            .animation(.easeInOut(duration: 0.25), value: appState.lightboxPhot
        }
    }
}
```

16. Formula Engine (Port from TypeScript)

Create `Utilities/FormulaEngine.swift`. This is a direct port of `src/formula.ts`.

Swift

```
import Foundation

struct FormulaEngine {

    // MARK: - Category Display Names

    static let categoryDisplay: [String: String] = [
        "PORTRAIT": "Face", "AUTOMOTIVE": "Car", "NIGHTLIFE": "Night",
        "ART": "Museum", "FITNESS": "Gym", "ABSTRACT": "Abstract",
        "FASHION": "Style", "ARCHITECTURE": "Space", "TRAVEL": "Travel",
        "DINING": "Eats", "WATCH": "Watch", "LIFESTYLE": "Life",
        "SCENE": "Scene", "STUDIO": "Studio"
    ]

    // MARK: - Slot Roles

    enum SlotRole: String, CaseIterable {
        case hook, contrast, detail, fashion, culture, watch
        case secondCar = "second-car"
        case insider, atmosphere
        case secondFashion = "second-fashion"
        case wildcard, closer
    }
```

```
    static let slotLabels: [SlotRole: String] = [
        .hook: "THE HOOK", .contrast: "THE CONTRAST", .detail: "THE DETAIL",
        .fashion: "THE FASHION BEAT", .culture: "THE CULTURAL MOMENT",
        .watch: "THE WATCH", .secondCar: "THE SECOND CAR",
        .insider: "THE INSIDER", .atmosphere: "THE ATMOSPHERE",
        .secondFashion: "SECOND FASHION BEAT", .wildcard: "THE WILDCARD",
        .closer: "THE CLOSER"
    ]

    static let template12: [SlotRole] = [
        .hook, .contrast, .detail, .fashion, .culture, .watch,
        .secondCar, .insider, .atmosphere, .secondFashion, .wildcard, .closer
    ]

    static let template7: [SlotRole] = [
        .hook, .contrast, .detail, .fashion, .culture, .secondCar, .closer
    ]

    // MARK: - Slot Scores

    static let slotScores: [SlotRole: [String: Int]] = [
        .hook:          ["PORTRAIT": 10, "AUTOMOTIVE": 10, "ART": 7, "FASHION":
        .contrast:      ["AUTOMOTIVE": 10, "PORTRAIT": 8, "ARCHITECTURE": 6],
        .detail:        ["WATCH": 10, "FASHION": 8, "AUTOMOTIVE": 6, "PORTRAIT"
        .fashion:       ["FASHION": 10, "PORTRAIT": 6, "LIFESTYLE": 4],
        .culture:       ["ART": 10, "PORTRAIT": 7, "LIFESTYLE": 5],
        .watch:         ["WATCH": 10, "FASHION": 5],
        .secondCar:     ["AUTOMOTIVE": 10, "ARCHITECTURE": 5],
        .insider:       ["PORTRAIT": 10, "NIGHTLIFE": 7, "LIFESTYLE": 5],
        .atmosphere:    ["ARCHITECTURE": 10, "TRAVEL": 10, "NIGHTLIFE": 8, "SCE
        .secondFashion: ["FASHION": 10, "PORTRAIT": 6],
        .wildcard:      ["DINING": 10, "ART": 8, "ARCHITECTURE": 7, "TRAVEL": 7
        .closer:        ["ARCHITECTURE": 10, "TRAVEL": 10, "NIGHTLIFE": 8, "ART
    ]

    // MARK: - Scoring

    static func hookScore(for photo: DumpPhoto) -> Int {
        let cat = photo.category.uppercased()
        switch cat {
        case "PORTRAIT": return 10
        case "AUTOMOTIVE": return 10
        case "ART": return 7
        case "NIGHTLIFE": return 6
        case "FASHION": return 5
        default: return 3
        }
    }
```

```
    static func scoreForSlot(_ photo: DumpPhoto, slot: SlotRole) -> Int {
        let cat = photo.category.uppercased()
        return slotScores[slot]?[cat] ?? 2
    }

    // MARK: - Arrange Photos

    static func arrangePhotos(_ photos: [DumpPhoto]) -> [DumpPhoto] {
        guard !photos.isEmpty else { return [] }
        let template = photos.count >= 10 ? template12 : template7
        let slots = Array(template.prefix(photos.count))

        guard slots.count > 1 else { return [photos[0]] }

        let hook = slots[0]
        let closer = slots[slots.count - 1]
        var middle = Array(slots[1..<(slots.count - 1)])
        middle.shuffle()
        let varied = [hook] + middle + [closer]

        var remaining = photos
        var result: [DumpPhoto?] = Array(repeating: nil, count: varied.count)

        for (slotIdx, slot) in varied.enumerated() {
            guard !remaining.isEmpty else { break }
            var bestIdx = 0
            var bestScore = -1
            for (i, photo) in remaining.enumerated() {
                let s = scoreForSlot(photo, slot: slot)
                if s > bestScore { bestScore = s; bestIdx = i }
            }
            result[slotIdx] = remaining[bestIdx]
            remaining.remove(at: bestIdx)
        }

        return result.compactMap { $0 }
    }

    // MARK: - Get Slot Role

    static func getSlotRole(index: Int, total: Int) -> SlotRole? {
        let template = total >= 10 ? template12 : template7
        guard index < template.count else { return nil }
        return template[index]
    }

    // MARK: - Vibe Check (Color Temperature Consistency)
```

```
    static func checkColorTemp(_ photos: [DumpPhoto]) -> Bool {
        guard photos.count >= 3 else { return true }
        let warm = photos.filter { ["TRAVEL", "DINING", "FITNESS"].contains($0.
        let cool = photos.filter { ["NIGHTLIFE", "ARCHITECTURE", "STUDIO"].cont
        let warmRatio = Double(warm) / Double(photos.count)
        let coolRatio = Double(cool) / Double(photos.count)
        return !(warmRatio > 0.25 && coolRatio > 0.25)
    }

    // MARK: - Generate Dump Title

    static func generateDumpTitle(for photos: [DumpPhoto]) -> String {
        guard !photos.isEmpty else { return "New Dump" }

        var counts: [String: Int] = [:]
        for p in photos { counts[p.category.uppercased(), default: 0] += 1 }
        let sorted = counts.sorted { $0.value > $1.value }
        let primary = sorted.first?.key ?? "LIFESTYLE"
        let secondary = sorted.count > 1 ? sorted[1].key : nil

        func pick(_ arr: [String]) -> String { arr.randomElement() ?? "New Dump

        // Multi-category combos
        if primary == "NIGHTLIFE" && secondary == "AUTOMOTIVE" {
            return pick(["Night Drives", "After Midnight", "The Late Run", "Cit
        }
        if primary == "AUTOMOTIVE" && secondary == "FASHION" {
            return pick(["Dressed to Drive", "The Style Run", "The Flex", "Movi
        }
        if primary == "PORTRAIT" && secondary == "NIGHTLIFE" {
            return pick(["Night Faces", "Who Was There", "The Room", "Present"]
        }
        if primary == "FASHION" && secondary == "ARCHITECTURE" {
            return pick(["The Location Scout", "Sharp Angles", "The Set", "Buil
        }

        switch primary {
        case "NIGHTLIFE":     return pick(["After Hours", "Nightfall", "Dark Ho
        case "AUTOMOTIVE":    return pick(["The Drive", "On The Road", "Moving"
        case "FASHION":       return pick(["The Fit", "Dressed", "The Look", "S
        case "ART":           return pick(["Culture Drop", "The Gallery", "Stud
        case "PORTRAIT":      return pick(["Faces", "Present", "In Frame", "The
        case "TRAVEL":        return pick(["On Location", "Away", "Out There",
        case "FITNESS":       return pick(["Work Mode", "Locked In", "Session",
        case "ARCHITECTURE":  return pick(["The Space", "Built", "Interiors", "
        case "DINING":        return pick(["The Table", "Last Night", "Good Eve
        case "WATCH":         return pick(["On The Wrist", "Time", "The Piece",
```

```
        case "STUDIO":        return pick(["In The Studio", "Session", "The Wor
        default:              return pick(["The Edit", "Recent", "What Happened
        }
    }

    // MARK: - Category Guesser (from filename)

    static func guessCategory(filename: String) -> String {
        let f = filename.lowercased()
        if f.contains("portrait") || f.contains("selfie") || f.contains("face")
        if f.contains("car") || f.contains("auto") || f.contains("bmw") || f.co
        if f.contains("studio") || f.contains("ssl") || f.contains("mix") { ret
        if f.contains("night") || f.contains("club") || f.contains("bar") { ret
        if f.contains("gym") || f.contains("fit") { return "FITNESS" }
        if f.contains("art") || f.contains("museum") || f.contains("gallery") {
        if f.contains("arch") || f.contains("hotel") || f.contains("build") { r
        if f.contains("travel") || f.contains("beach") || f.contains("miami") {
        if f.contains("fashion") || f.contains("gucci") || f.contains("balenc")
        if f.contains("watch") || f.contains("rm") || f.contains("patek") || f.
        return "LIFESTYLE"
    }

    static func detectHuji(filename: String) -> Bool {
        let f = filename.lowercased()
        return f.contains("huji") || f.contains("dexp") || f.contains("film") |
    }
}
```

---

17. Undo/Redo System

Create `Utilities/UndoManager.swift`.

Swift

```
import Foundation

@MainActor
final class DumpsterUndoManager: ObservableObject {
    struct Snapshot {
        let photoIDs: [String]          // All DumpPhoto IDs at snapshot time
        let dumps: [(id: String, num: Int, title: String, photoIDs: [String])]
        let timestamp: Date
    }

    private var undoStack: [Snapshot] = []
    private var redoStack: [Snapshot] = []
```

```
    private let maxSnapshots = 200

    var canUndo: Bool { !undoStack.isEmpty }
    var canRedo: Bool { !redoStack.isEmpty }

    /// Call BEFORE any mutation to save current state.
    func pushSnapshot(photos: [DumpPhoto], dumps: [PhotoDump]) {
        let snap = Snapshot(
            photoIDs: photos.map { $0.id },
            dumps: dumps.map { ($0.id, $0.num, $0.title, $0.photoIDs) },
            timestamp: Date()
        )
        undoStack.append(snap)
        redoStack.removeAll()
        if undoStack.count > maxSnapshots { undoStack.removeFirst() }
    }

    func popUndo() -> Snapshot? {
        undoStack.popLast()
    }

    func pushRedo(_ snapshot: Snapshot) {
        redoStack.append(snapshot)
    }

    func popRedo() -> Snapshot? {
        redoStack.popLast()
    }

    /// Clear history older than 24 hours.
    func pruneOldSnapshots() {
        let cutoff = Date().addingTimeInterval(-86400)
        undoStack.removeAll { $0.timestamp < cutoff }
        redoStack.removeAll { $0.timestamp < cutoff }
    }
}
```

Integration: The `MainAppView` should hold `@StateObject var undoManager = DumpsterUndoManager()` and pass it via `.environmentObject()`. Every mutation function (add photo, remove photo, reorder, create dump, delete dump) must call `undoManager.pushSnapshot(...)` before making changes.

---

18. JavaScript Bridge Replacement Map

This section maps every JS bridge message handler to its native replacement.

18.1. Message Handlers (Web → Native)

These are the `WKScriptMessageHandler` cases in `ContentView.Coordinator.handleBridgeMessage`:

Bridge Message Type	Current Behavior	Native Replacement
`"openAISuggest"`	Sets `appState.showAISuggest = true`	**No change needed** — the native button/gesture already sets this. Remove the bridge handler.
`"openFileCabinet"`	Sets `appState.showFileCabinet = true`	**No change needed** — the Dynamic Island long-press and menu button already set this. Remove the bridge handler.
`"bridgeReady"`	Logs readiness, sends cached captions to web	**Delete entirely.** No web bridge exists anymore.
`"dumpCount"`	Updates `appState.dumpCount` from web's `localStorage`	**Replace** with a SwiftData query: `appState.dumpCount = fetchDumps().count`. Update this whenever dumps are created/deleted.
`"requestCaptions"`	Opens AI suggest panel	**Replace** with a native button that sets `appState.showAISuggest = true`.

18.2. Message Handlers (Native → Web)

These are `appState.evaluateJS(...)` calls:

JS Call	Current Purpose	Native Replacement
`window.__dumpsterBridge.receiveCaptions(json)`	Sends LLM caption results to the web app	**Delete.** Captions are now stored as `DumpCaption` records in SwiftData and displayed by `CaptionPoolView` / `CabinetMenuView`.
`window.__dumpsterBridge.receiveStatus(status)`	Sends status text to the web app	**Delete.** Status is shown in the Dynamic Island capsule natively.

`window.__dumpsterAI.createDumps(json)`	Creates dumps from Vision AI clusters in the web store	**Replace** with direct SwiftData insertion in `AISuggestView.createDumps()`. See Section 19.
`localStorage.getItem('dumpster-dumps')` (in `webView(_:didFinish:)`)	Reads dump count after page load	**Delete.** Dump count comes from SwiftData query.

18.3. Injected Scripts

The `webFixScript` `WKUserScript` in `DumpsterWebView.makeUIView` injects CSS fixes and sets up `window.__dumpsterBridge`. **Delete the entire script.** None of it is relevant to native views.

---

19. AISuggestView Integration Changes

19.1. Remove WebKit Import

Delete `import WebKit` from the top of `AISuggestView.swift`.

19.2. Add Environment

Add `@Environment(\.modelContext) private var modelContext` to `AISuggestView`.

19.3. Replace `createDumps()`

Replace the current `createDumps()` method (which serializes to JSON and calls `appState.evaluateJS`) with:

Swift

```
private func createDumps() {
    isCreating = true
    appState.showStatus("Creating dumps...", duration: 10)

    let selectedGroups = clusters.enumerated()
        .filter { selectedClusters.contains($0.offset) }
        .map { $0.element }

    let storageManager = PhotoStorageManager.shared

    for cluster in selectedGroups {
        var photoIDs: [String] = []
```

```
        for analyzedPhoto in cluster.photos {
            // Save the thumbnail to permanent storage
            let relativePath = storageManager.saveImage(
                analyzedPhoto.image,
                filename: analyzedPhoto.filename
            )

            let newPhoto = DumpPhoto(
                localPath: relativePath,
                filename: analyzedPhoto.filename,
                category: analyzedPhoto.category,
                labels: analyzedPhoto.labels,
                isHuji: false
            )
            modelContext.insert(newPhoto)
            photoIDs.append(newPhoto.id)
        }

        // Determine next dump number
        let existingDumps = (try? modelContext.fetch(FetchDescriptor<PhotoDump>
        let nextNum = (existingDumps.map { $0.num }.max() ?? 0) + 1

        let newDump = PhotoDump(
            num: nextNum,
            title: cluster.title,
            photoIDs: photoIDs
        )
        modelContext.insert(newDump)

        // Store matching captions
        if let captionResult = appState.captionResults.first(where: { $0.dumpTi
            for captionText in captionResult.captions {
                let caption = DumpCaption(
                    text: captionText,
                    style: "clean",
                    dumpId: newDump.id
                )
                modelContext.insert(caption)
            }
        }
    }

    try? modelContext.save()

    appState.dumpCount += selectedGroups.count
    appState.showStatus("Dumps created!", duration: 3)
    isPresented = false
```

```
    isCreating = false
}
```

19.4. Remove `sendCaptionsToWeb` Calls

In `generateCaptionsForClusters`, remove the line `appState.sendCaptionsToWeb(results)`. The captions are stored in `appState.captionResults` and will be consumed by the native `CabinetMenuView` (which already reads from `appState.captionResults`).

---

20. DumpsterApp Entry Point Changes

Swift

```swift
import SwiftUI
import SwiftData

@main
struct DumpsterApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
                .ignoresSafeArea()
        }
        .modelContainer(for: [DumpPhoto.self, PhotoDump.self, DumpCaption.self]
    }
}
```

---

21. ContentView Rewrite

The rewritten `ContentView.swift` retains the Dynamic Island, overlays, and `AppState`, but replaces `DumpsterWebView` with `MainAppView`.

21.1. Imports

Swift

```swift
import SwiftUI
import SwiftData
// Remove: import WebKit
```

21.2. AppState Changes

See Section 8. Remove `webView`, `evaluateJS`, `sendCaptionsToWeb`. Keep everything else.

21.3. ContentView Body

Swift

```
struct ContentView: View {
    @StateObject private var appState = AppState()
    @StateObject private var undoManager = DumpsterUndoManager()
    @State private var isExpanded = false
    private let impact = UIImpactFeedbackGenerator(style: .medium)

    var body: some View {
        ZStack(alignment: .top) {
            // 1. NATIVE CONTENT (replaces DumpsterWebView)
            MainAppView()
                .environmentObject(appState)
                .environmentObject(undoManager)

            // 2. DYNAMIC ISLAND (unchanged from current implementation)
            dynamicIslandView
                .frame(maxWidth: .infinity)
                .padding(.top, 13)
                .ignoresSafeArea(.all)
                .animation(.easeInOut(duration: 2.5), value: isExpanded)
                .onTapGesture { /* ... same as current ... */ }
                .onLongPressGesture(minimumDuration: 0.5) { /* ... same ... */

            // 3. AI SUGGESTIONS (unchanged)
            if appState.showAISuggest {
                CabinetMenuView(isPresented: $appState.showAISuggest, appState:
                    .zIndex(10)
                    .transition(.move(edge: .bottom))
            }

            // 4. FILE CABINET (unchanged)
            if appState.showFileCabinet {
                FileCabinetMenuView(isPresented: $appState.showFileCabinet, app
                    .zIndex(20)
                    .transition(.opacity.combined(with: .scale(scale: 0.95)))
            }

            // 5. LIGHTBOX (new)
            LightboxView()
                .environmentObject(appState)
                .zIndex(30)
        }
```

```
        .background(Color.black)
        .preferredColorScheme(.dark)
        .fullScreenCover(isPresented: $appState.showSettings) {
            SettingsView(isPresented: $appState.showSettings)
        }
        .onAppear {
            DispatchQueue.main.asyncAfter(deadline: .now() + 5.0) { isExpanded
        }
    }

    // dynamicIslandView — KEEP EXACTLY AS-IS from current ContentView
    private var dynamicIslandView: some View { /* ... unchanged ... */ }
}
```

21.4. Keep These Components in ContentView.swift

The following structs currently defined in `ContentView.swift` should remain (they are already native):

- `CabinetMenuView` (AI suggestions bottom sheet)
- `CaptionOptionButton`
- `Color(hex:)` extension

21.5. Remove These Components from ContentView.swift

- `DumpsterWebView` struct (entire `UIViewRepresentable` + `Coordinator`)

---

22. Pending Features — Full Specifications

22.1. Tap to Rename Dump

Already covered in DumpCardView (Section 12.2). The title is a `TextField` that activates on tap.

22.2. Caption Pool

Already covered in Section 14.

22.3. Generate Captions Per Dump

On each `DumpCardView`, add a sparkle button. On tap:

1. Resolve the dump's photos to get categories and labels.
2. Build a `LLMService.CaptionRequest`.

3. Call `LLMService.shared.generateCaptions(for:)`.
4. Present results in a `DumpCaptionBubble` — a floating overlay anchored to the dump card, showing 10 scrollable caption options. Each option is tappable to copy. Dismiss by tapping outside or swiping down.

22.4. Undo/Redo

Covered in Section 17. Display undo (left arrow) and redo (right arrow) buttons in the header. Style: 32x32 rounded rectangles, disabled state at 40% opacity.

22.5. Auto-Create Dump Button

The "Auto-Generate" button in the action row. On tap, show a popover/sheet:

- **Inspiration field:** Optional text input ("What's the vibe?")
- **Photo count picker:** Stepper from 2–20, default 7.
- **Style selector:** "Day" / "Night" / "Freestyle" segmented control.
- **"Generate Dump" button:** Calls `FormulaEngine.arrangePhotos()` on available pool photos, creates a new `PhotoDump`, and scrolls to it.

22.6. Photo Labels

In the photo "..." menu, add "Add Label" option. On tap, show a `TextField` alert. Auto-suggest labels based on the photo's Vision AI labels (from `PhotoAnalyzer`). Multiple labels per photo. Labels appear as small chips below the photo card in pool view. Tappable to edit or remove.

22.7. Magnifying Glass Search

In the pool header, a magnifying glass icon toggles a search `TextField`. Bound to `appState.poolSearchQuery`. The pool grid filters in real-time by matching against `filename`, `category`, and `labels`.

22.8. Hamburger Filter Menu

Create `Views/PoolFilterMenu.swift`. Triggered by the hamburger icon in the pool header. Slides in from the right as an overlay panel. Contains toggle rows for each filter: Huji, Videos, Starred, Used. Each row has a label and a toggle switch. Scrolling through the list triggers `UISelectionFeedbackGenerator().selectionChanged()` for haptic feedback.

22.9. Share/Export Button

On each `DumpCardView`, a share button. On tap:

1. Resolve the dump's photos in order.
2. Load each `UIImage` from disk.
3. Present `UIActivityViewController` (wrapped in `UIViewControllerRepresentable`) with the image array.
4. Images are shared in dump order (first = hook, last = closer).

22.10. "Add Photos" Upload Card

At the end of the photo pool grid. A dashed-border card with a camera/plus icon. On tap, presents `PhotosPicker` from `PhotosUI`. Selected photos are saved via `PhotoStorageManager` and inserted as `DumpPhoto` records.

22.11. Confirm Dialog Before "Clear All Dumps"

Wherever "Clear All" or "Reset" is triggered (e.g., in FileCabinetMenuView or settings), wrap it with:

Swift

```
.confirmationDialog("Clear all dumps?", isPresented: $showClearConfirm) {
    Button("Clear All Dumps", role: .destructive) { clearAllDumps() }
    Button("Cancel", role: .cancel) {}
} message: {
    Text("This will remove all dumps. Photos will remain in your pool.")
}
```

22.12. Pinch/Slider to Resize Pool Photos

Add a `MagnificationGesture` to the `PhotoPoolView` grid:

Swift

```
.gesture(
    MagnificationGesture()
        .onEnded { scale in
            if scale > 1.3 {
                // Zoom in → fewer columns → larger photos
                appState.poolSize = appState.poolSize == .small ? .medium : .la
            } else if scale < 0.7 {
                // Zoom out → more columns → smaller photos
                appState.poolSize = appState.poolSize == .large ? .medium : .sm
            }
```

```
        }
)
```

22.13. Vibration Haptics on Filter Scroll

In `PoolFilterMenu`, use a `ScrollView` with a `GeometryReader` to detect when each filter row crosses the center of the visible area. Trigger `UISelectionFeedbackGenerator().selectionChanged()` at each crossing.

---

23. Animations & Transitions Reference

Action	Animation
Dynamic Island expand/collapse	`.interpolatingSpring(stiffness: 120, damping: 12)`
File Cabinet appear	`.easeOut(duration: 0.4)` for backdrop, `.spring(response: 0.6, dampingFraction: 0.8)` for tabs
File Cabinet tab open	`.spring(response: 0.45, dampingFraction: 0.82)`
AI Suggestions sheet dismiss	`.spring(response: 0.4, dampingFraction: 0.8)`
Remove photo from dump	Scale to 0.8 + fade out over 0.3s, remaining cards slide with `.spring()`
Lightbox appear/dismiss	`.easeInOut(duration: 0.25)` opacity
Pool filter menu slide	`.spring(response: 0.35, dampingFraction: 0.85)`
Photo card selection toggle	`.easeInOut(duration: 0.15)` border color change
Dump card delete	Scale to 0.95 + fade out over 0.3s, then remove
Pool toggle pill slide	`.spring(response: 0.35, dampingFraction: 0.8)` translateX
Undo/redo button press	`.easeInOut(duration: 0.1)` scale to 0.95

---

24. Haptic Feedback Map

Action	Haptic Type

Tap Dynamic Island	`UIImpactFeedbackGenerator(style: .medium)`
Long-press Dynamic Island	`UIImpactFeedbackGenerator(style: .medium)`
Open File Cabinet tab	`UIImpactFeedbackGenerator(style: .medium)`
Select/deselect AI cluster	`UIImpactFeedbackGenerator(style: .medium, intensity: 0.6)`
Tap photo in pool (selection mode)	`UIImpactFeedbackGenerator(style: .light)`
Copy caption to clipboard	`UIImpactFeedbackGenerator(style: .light)`
Drag photo (start)	`UIImpactFeedbackGenerator(style: .medium)`
Drop photo (reorder complete)	`UIImpactFeedbackGenerator(style: .light)`
Filter scroll crossing	`UISelectionFeedbackGenerator().selectionChanged()`
Delete dump	`UINotificationFeedbackGenerator().notificationOccurred(.warning)`
Create dump success	`UINotificationFeedbackGenerator().notificationOccurred(.success)`

---

25. Photo Storage Strategy

Create `Utilities/PhotoStorageManager.swift`.

Swift

```
import UIKit

final class PhotoStorageManager {
    static let shared = PhotoStorageManager()

    private let photosDir: URL = {
        let docs = FileManager.default.urls(for: .documentDirectory, in: .userD
        let dir = docs.appendingPathComponent("photos", isDirectory: true)
        try? FileManager.default.createDirectory(at: dir, withIntermediateDirec
        return dir
    }()

    /// Save a UIImage to the photos directory. Returns the relative path (e.g.
    func saveImage(_ image: UIImage, filename: String? = nil) -> String {
```

```
        let name = filename ?? "\(UUID().uuidString).jpg"
        // Ensure unique filename
        let uniqueName: String
        if FileManager.default.fileExists(atPath: photosDir.appendingPathCompon
            uniqueName = "\(UUID().uuidString.prefix(8))_\(name)"
        } else {
            uniqueName = name
        }

        let fileURL = photosDir.appendingPathComponent(uniqueName)
        if let data = image.jpegData(compressionQuality: 0.85) {
            try? data.write(to: fileURL)
        }
        return "photos/\(uniqueName)"
    }

    /// Load a UIImage from a relative path.
    func loadImage(relativePath: String) -> UIImage? {
        let docs = FileManager.default.urls(for: .documentDirectory, in: .userD
        let url = docs.appendingPathComponent(relativePath)
        return UIImage(contentsOfFile: url.path)
    }

    /// Delete a photo file.
    func deleteImage(relativePath: String) {
        let docs = FileManager.default.urls(for: .documentDirectory, in: .userD
        let url = docs.appendingPathComponent(relativePath)
        try? FileManager.default.removeItem(at: url)
    }
}
```

When importing photos from `PhotosPicker`:

1. Load the `Data` from `PhotosPickerItem`.
2. Create a `UIImage`.
3. Call `PhotoStorageManager.shared.saveImage(image, filename:)`.
4. Create a `DumpPhoto` record with the returned `localPath`.
5. Insert into SwiftData `ModelContext`.

---

26. Xcode Project Changes

26.1. Remove from Build Phases

- **Copy Bundle Resources:** Remove the `dist` folder reference.

- **Link Binary With Libraries:** Remove `WebKit.framework`. Keep `Vision.framework`.

26.2. Add New Files to Target

Add all new `.swift` files listed in Section 6 to the `DumpsterIOS` target.

26.3. Folder Structure

Organize the Xcode project navigator:

```
DumpsterIOS/
├── DumpsterApp.swift
├── ContentView.swift
├── Models/
│   ├── DumpPhoto.swift
│   ├── PhotoDump.swift
│   └── DumpCaption.swift
├── Views/
│   ├── MainAppView.swift
│   ├── DumpCardView.swift
│   ├── PhotoCardView.swift
│   ├── PhotoPoolView.swift
│   ├── CaptionPoolView.swift
│   ├── LightboxView.swift
│   ├── PoolFilterMenu.swift
│   └── DumpCaptionBubble.swift
├── Utilities/
│   ├── FormulaEngine.swift
│   ├── UndoManager.swift
│   ├── PhotoStorageManager.swift
│   └── Theme.swift
├── Preserved/ (existing native files — no changes)
│   ├── LLMService.swift
│   ├── CaptionService.swift
│   ├── CreditManager.swift
│   ├── PaywallView.swift
│   ├── PhotoAnalyzer.swift
│   ├── FileCabinetMenuView.swift
│   ├── SettingsView.swift
│   └── AISuggestView.swift (modified per Section 19)
└── Assets.xcassets/
```

---

27. Step-by-Step Execution Order

Execute these steps in order. Each step should result in a compilable project.

Step 1 — Data Layer:

Create `DumpPhoto.swift`, `PhotoDump.swift`, `DumpCaption.swift` in a `Models/` group. Create `PhotoStorageManager.swift` and `Theme.swift` in a `Utilities/` group. Update `DumpsterApp.swift` to configure the `ModelContainer`.

Step 2 — State Management:

Rewrite `AppState` in `ContentView.swift` per Section 8. Remove `webView`, `evaluateJS`, `sendCaptionsToWeb`. Add the new UI state properties. Create `DumpsterUndoManager.swift`.

Step 3 — Remove Web Layer:

Delete `DumpsterSchemeHandler.swift`. Delete the `dist/` folder reference from the Xcode project. Remove `WebKit.framework` from linked frameworks. Remove `import WebKit` from `ContentView.swift` and `AISuggestView.swift`. Delete the `DumpsterWebView` struct and its `Coordinator` from `ContentView.swift`.

Step 4 — Formula Engine:

Create `FormulaEngine.swift` with the full port from `formula.ts` (Section 16).

Step 5 — PhotoCardView:

Create `PhotoCardView.swift` per Section 11. Implement all visual states, gesture handling, and the "..." menu.

Step 6 — DumpCardView:

Create `DumpCardView.swift` per Section 12. Implement the header, horizontal carousel, drag-to-reorder, "+" card, progress bar, and share button.

Step 7 — PhotoPoolView:

Create `PhotoPoolView.swift` per Section 13. Implement the grid, filtering, search, size toggle, upload card, and add-to-dump selection mode.

Step 8 — CaptionPoolView:

Create `CaptionPoolView.swift` per Section 14.

Step 9 — LightboxView:

Create `LightboxView.swift` per Section 15.

Step 10 — MainAppView:

Create `MainAppView.swift` that assembles the header, dump list, action row, pool toggle, and pool area into a single scrollable view per Section 10.

Step 11 — ContentView Assembly:

Update `ContentView.swift` per Section 21. Replace `DumpsterWebView` with `MainAppView`. Keep the Dynamic Island, overlays, and all existing native components.

Step 12 — AISuggestView Integration:

Modify `AISuggestView.swift` per Section 19. Replace the JS bridge `createDumps()` with direct SwiftData insertion. Remove `import WebKit`.

Step 13 — Pending Features:

Implement the pending features per Section 22: rename dump, caption generation bubble, undo/redo buttons, auto-create dump sheet, photo labels, search, filter menu, share/export, clear confirmation, pinch-to-resize, haptics.

Step 14 — Polish:

Apply all animations per Section 23. Apply all haptics per Section 24. Test dark/day mode switching. Test persistence across app launches. Verify the Dynamic Island capsule still works correctly. Verify `FileCabinetMenuView`, `PaywallView`, and `CreditBadge` still function.

Appendix A: UserDefaults Keys (Preserved)

These keys are used by the existing native code and must not be changed:

Key	Used By	Purpose
`llm_api_key_openai`	LLMService	OpenAI API key
`llm_api_key_claude`	LLMService	Claude API key
`llm_api_key_gemini`	LLMService	Gemini API key
`llm_api_key_manus`	LLMService	Manus API key
`llm_api_key_perplexity`	LLMService	Perplexity API key
`llm_model_openai`	LLMService	Selected OpenAI model
`llm_model_claude`	LLMService	Selected Claude model
`llm_model_gemini`	LLMService	Selected Gemini model
`llm_model_manus`	LLMService	Selected Manus model
`llm_model_perplexity`	LLMService	Selected Perplexity model
`llm_labeling_sensitivity`	LLMService	Labeling confidence threshold (0.0–1.0)

`llm_caption_style`	LLMService	Caption style preference
`llm_intelligence_level`	LLMService	Auto-dump intelligence level
`dumpster_credit_balance`	CreditManager	Credit balance
`dumpster_credits_onboarded`	CreditManager	First-launch flag
`dumpster_colorMode`	AppState (new)	Color mode preference
`dumpster_poolSize`	AppState (new)	Pool size preference

Appendix B: Constraints & Rules

- **Max photos per dump:** 20
- **Sweet spot:** 4–8 photos per dump
- **Peak zone (progress bar gold):** 10–12 photos
- **Undo stack max:** 200 snapshots
- **Undo history TTL:** 24 hours
- **Photo thumbnail max dimension (AI analysis):** 512px
- **Vision confidence threshold:** 0.10
- **Max clusters from PhotoAnalyzer:** 5
- **Max photos per cluster:** 20
- **PhotosPicker max selection (AI Builder):** 100
- **LLM request timeout:** 30 seconds
- **Caption max length:** 150 characters
- **Title max length:** 30 characters
- **Free starter credits:** 10
- **Credit cost per AI action:** 1

Appendix C: Color Hex Reference

Token	Dark Mode	Day Mode
`bg`	`#0A0A0A`	`#F0EBE0`

bg1	#111111	#E8E2D5
bg2	#181818	#DFD8C8
bg3	#222222	#D4CCBA
border	#1E1E1E	#C8BFA8
border2	#2A2A2A	#B8AE98
border3	#333333	#A89E88
text	#E8E8E8	#1A1610
text2	#999999	#5A5040
text3	#555555	#8A7A60
gold	#C8A96E	#C8A96E
menu-bg	rgba(20,20,20,0.75)	rgba(230,224,210,0.75)

End of specification.